

CITYFLOW

Cahier des charges du projet fil rouge

Plateforme de mobilité urbaine — Architecture de persistance polyglotte

Module	NoSQL — B3 INFO Tronc Commun
Formateur	KAKE Abdoulaye
Établissement	Ynov Campus Lyon
Année	2025 / 2026
Durée projet	3 journées (intégré aux séances de cours)
Format équipe	3 à 4 étudiants
Pondération évaluation	Livrable 50% + Démo 50%

1. Contexte et vision du projet

1.1. Le pitch

CityFlow est une startup fictive qui développe une plateforme unifiée de mobilité urbaine pour la métropole de Lyon. L'idée est simple : permettre à un utilisateur de planifier, réserver et suivre un trajet multimodal combinant transports en commun, covoiturage et vélos en libre-service, le tout depuis une seule application.

La direction technique a fait un choix stratégique : adopter une **architecture de persistance polyglotte**, c'est-à-dire utiliser plusieurs bases de données spécialisées plutôt qu'une seule base relationnelle généraliste. Chaque type de donnée est stocké dans la base la mieux adaptée à son usage. Votre équipe est chargée de concevoir et implémenter cette architecture en utilisant les quatre bases NoSQL étudiées dans le module : MongoDB, Redis, Cassandra et Neo4j.

1.2. Pourquoi une architecture polyglotte ?

Une architecture polyglotte (terme popularisé par Martin Fowler) consiste à utiliser plusieurs technologies de stockage adaptées à chaque besoin, plutôt que de forcer une base unique à tout faire. C'est un choix courant dans les grandes entreprises tech (Netflix, Uber, Airbnb) mais il introduit aussi de la complexité opérationnelle qu'il faut savoir justifier.

Le projet CityFlow vous force à **justifier chaque choix technique** : pourquoi cette donnée dans cette base ? Quel serait le coût d'utiliser une seule base relationnelle à la place ? Quels trade-offs acceptez-vous ? Ces questions seront évaluées lors de la démonstration.

1.3. Périmètre fonctionnel

Le périmètre est volontairement **simplifié** : il ne s'agit pas de livrer une application end-to-end avec frontend, mais de produire l'**architecture de données** et les **requêtes représentatives** qui démontrent que chaque base est utilisée à bon escient. Pas de développement d'API REST, pas d'UI : on se concentre sur la modélisation, les schémas, les requêtes et l'orchestration via Docker Compose.

2. Architecture cible — répartition des données

Chaque base de données a un rôle dédié dans l'architecture. Le tableau ci-dessous synthétise les responsabilités. Vous devrez justifier ces choix lors de la démonstration.

Base	Rôle dans CityFlow	Justification du choix
MongoDB	Profils utilisateurs, trajets effectués, véhicules disponibles	Schémas flexibles, données riches et imbriquées (un trajet = utilisateur + véhicule + étapes), requêtes ad-hoc fréquentes
Redis	Cache des disponibilités de stations, sessions utilisateurs, leaderboard des conducteurs	Latence sub-milliseconde indispensable pour le temps réel, expiration automatique, structures spécialisées (sorted sets pour les classements)

Cassandra	Historique des passages aux stations (événements horodatés), logs de connexions	Volume massif d'écritures (1 mesure/min × 1000+ stations), pattern time-series, scalabilité horizontale
Neo4j	Réseau de transport (stations, lignes, connexions), calcul d'itinéraires	Modèle graphe naturel pour un réseau, requêtes de plus court chemin, traversées multi-niveaux

*

***Note importante** : dans une vraie application, les bases ne sont pas isolées — elles communiquent via des services applicatifs. Pour ce projet, nous ne développons pas la couche applicative. Chaque base contient ses propres données, et les requêtes que vous écrivez démontrent le rôle de chacune dans le système global.*

3. User stories — les fonctionnalités à supporter

Voici les user stories que votre architecture doit pouvoir supporter. Pour chacune, identifiez quelle(s) base(s) vous interrogez et démontrez la requête correspondante lors de la démo.

3.1. Stories MongoDB — données métier riches

- **US-M1** : En tant qu'utilisateur, je veux pouvoir consulter mon profil et l'historique de mes 10 derniers trajets.
- **US-M2** : En tant qu'administrateur, je veux pouvoir lister tous les véhicules d'un type donné (vélo, voiture, scooter) disponibles dans un arrondissement.
- **US-M3** : En tant qu'analyste, je veux pouvoir obtenir des statistiques agrégées : nombre de trajets par jour, distance moyenne, top 5 des conducteurs.
- **US-M4** : En tant que développeur, je veux pouvoir effectuer une recherche full-text sur les commentaires laissés par les utilisateurs après leurs trajets.

3.2. Stories Redis — temps réel et performance

- **US-R1** : En tant qu'utilisateur, je veux voir en temps réel combien de vélos sont disponibles à la station X.
- **US-R2** : En tant qu'utilisateur, je veux que ma session reste active pendant 30 minutes après ma dernière action, sans rechargement.
- **US-R3** : En tant qu'utilisateur, je veux consulter le classement des 10 utilisateurs les plus actifs du mois.
- **US-R4** : En tant que système, je veux limiter chaque utilisateur à 100 requêtes API par minute (rate limiting).

3.3. Stories Cassandra — données massives et historique

- **US-C1** : En tant qu'analyste, je veux retrouver l'historique des passages d'une station spécifique sur une période donnée.
- **US-C2** : En tant que système, je veux pouvoir enregistrer plusieurs milliers d'événements de passage par minute sans dégradation.
- **US-C3** : En tant qu'utilisateur, je veux consulter mes propres connexions des 30 derniers jours (audit).
- **US-C4** : En tant qu'analyste, je veux obtenir l'évolution journalière du nombre de passages pour identifier les pics d'affluence.

3.4. Stories Neo4j — relations et itinéraires

- **US-N1** : En tant qu'utilisateur, je veux calculer le plus court chemin entre deux stations.
- **US-N2** : En tant qu'utilisateur, je veux trouver toutes les stations accessibles à moins de 15 minutes d'une station donnée.
- **US-N3** : En tant que planificateur, je veux identifier les stations « hubs » (les plus connectées du réseau).
- **US-N4** : En tant qu'utilisateur, je veux savoir s'il existe un itinéraire entre deux stations sans correspondance.

4. Livrables attendus

Tous les livrables sont rassemblés dans un **dépôt Git unique par équipe** (GitHub recommandé) dont l'URL doit être communiquée au formateur au plus tard à la fin de la séance 4.

4.1. Structure attendue du dépôt

Voici la structure de dépôt recommandée. Vous pouvez adapter, mais l'esprit doit être respecté :

```
cityflow/
III README.md                (présentation, équipe, lancement) III
docker-compose.yml           (orchestration des 4 bases)
III .env.example              (variables d'environnement type) III docs/
IIII architecture.md         (schéma global + justifications) IIII
modélisation-mongodb.md      (collections + choix embed/ref) IIII
modélisation-redis.md        (clés + structures + naming)
IIII modélisation-cassandra.md (tables + partition keys + justifs) IIII
modélisation-neo4j.md        (nœuds, relations, propriétés)
III seed/
IIII mongodb/init.js          (création collections + données) IIII redis/init.sh
(peuplement initial)
IIII cassandra/init.cql       (création keyspace + tables + données) IIII
neo4j/init.cypher             (création nœuds et relations)
III queries/
    III mongodb-queries.md     (requêtes user stories M1-M4)
    III redis-queries.md       (requêtes user stories R1-R4)
    III cassandra-queries.md   (requêtes user stories C1-C4)
    III neo4j-queries.md       (requêtes user stories N1-N4)
```

4.2. Détail du contenu attendu

README.md

Présentation du projet, composition de l'équipe, instructions de lancement (1 commande), captures d'écran, lien vers chaque document de modélisation. Ce fichier doit pouvoir être lu en 2 minutes et donner envie.

docker-compose.yml

Lance les 4 bases en une commande : `docker compose up -d`. Les ports doivent être exposés proprement, les volumes nommés pour persistance, les healthchecks configurés. Variables sensibles en `.env`.

Scripts de seed

Chaque base doit pouvoir être peuplée automatiquement au lancement du docker-compose, avec au moins 20-30 entités par modèle pour permettre des requêtes représentatives.

Documents de requêtes

Un par base. Pour chaque user story, donner la requête complète, son résultat attendu, et un commentaire expliquant le concept mis en œuvre (ex. : « cette requête démontre l'agrégation MongoDB »).

5. Critères de qualité — comment vous serez évalués

Le projet est évalué sur 3 dimensions complémentaires. Le détail des critères se trouve dans le document « Grille d'évaluation NoSQL B3 ».

Dimension	Poids	Ce qui est évalué
Livrable d'équipe	50%	Qualité du code, structure du dépôt, lisibilité des schémas, justification des choix, présence et qualité du README, scripts de seed fonctionnels
Démonstration technique	50%	Lancement réussi du docker-compose, démonstration de requêtes représentatives sur chaque base, capacité à expliquer les choix, réponse aux questions du formateur

I Avertissement : un projet techniquement fonctionnel mais sans justification écrite des choix sera **fortement pénalisé**. Le but n'est pas de copier-coller des requêtes mais de **démontrer que vous avez compris pourquoi** on utilise telle base pour telle donnée. Le « pourquoi » compte autant que le « comment ».

6. Ressources et documentation

Techno	Documentation officielle	Outil GUI recommandé
MongoDB	https://www.mongodb.com/docs/manual/	MongoDB Compass
Redis	https://redis.io/docs/	RedisInsight
Cassandra	https://cassandra.apache.org/doc/	cqlsh (CLI intégré)
Neo4j	https://neo4j.com/docs/	Neo4j Browser (intégré image Docker)

Formations gratuites recommandées

- **MongoDB University** : <https://learn.mongodb.com> — certifications gratuites M001 et M201
- **Redis University** : <https://university.redis.com> — parcours RU101 introduction
- **DataStax Academy** (Cassandra) : <https://www.datastax.com/learn> — cours gratuits sur le data modeling
- **Neo4j GraphAcademy** : <https://graphacademy.neo4j.com> — parcours complets gratuits avec certification

7. Rappel final — l'esprit du projet

Ce projet n'est pas un concours de tooling ou de lignes de code. Sa valeur réside dans votre **capacité à argumenter techniquement** : pourquoi cette base, pourquoi ce schéma, pourquoi cette requête. Un projet techniquement modeste mais brillamment argumenté vaut mieux qu'un projet ambitieux mais incompris.

La persistance polyglotte est un sujet de **maturité technique**. Elle s'utilise quand elle se justifie, et pas par effet de mode. Démontrez que vous comprenez ce trade-off, et vous aurez gagné la compétence essentielle de ce module : **savoir choisir la bonne base pour le bon besoin**.